

Contents

Revolut Merchant API. Payment Implementation Guide	1
1. How the pieces fit together	1
2. Order creation	1
3. Payment processing (between order creation and completion)	3
4. Capturing (manual capture mode)	4
5. Refunds	4
6. Webhooks	5
7. PCI compliant database schema	6
8. Error handling	8
9. Security practices beyond just PCI scope	8
10. Assumptions I made	9
References	9

Revolut Merchant API. Payment Implementation Guide

Author: Prem Kumar **Scope:** Order creation, payment processing, refunds. Plus DB schema, webhooks, and security. **API version used:** 2026-04-20 (Revolut Merchant API, versioned through a header)

1. How the pieces fit together

Revolut's Merchant API is built around one main object: the **Order**. You don't call a "charge card" endpoint directly like some older gateways do. You create an Order first, then a payment method (card field, Revolut Pay, Apple/Google Pay, or the hosted checkout page) pays into that order. A refund is actually a second Order, with type: refund, linked back to the original order through related_order_id. Once that clicks, the rest of the API makes a lot more sense.

There are three parts in a real integration:

1. **Your backend.** Creates orders, stores payment state, never touches raw card data.
2. **Revolut's checkout surface.** Either the hosted checkout page (you redirect the customer) or the embedded Merchant Web SDK (a card field rendered in an iframe on your own site). Either way, the card details go straight from the customer's browser to Revolut. Your server never sees them.
3. **Webhook listener.** This is your real source of truth for order status, because payment confirmation is not instant. There's 3DS challenges, bank delays, and so on.

Authentication is just a static bearer token. No OAuth flow needed:

```
Authorization: Bearer sk_1234567890ABCdefGHIjklMN0pqrSTUvwxyz...
Revolut-API-Version: 2026-04-20
```

The secret key stays on your server only. There's a separate public key for the frontend, used to set up the Merchant Web SDK. Every money related request also needs the Revolut-API-Version header. Skip it and the request just gets rejected. I've pinned 2026-04-20 here since it's the latest stable version at the time of writing.

2. Order creation

Endpoint: POST /api/orders

This always has to be a server to server call. It should never be triggered from the frontend, because it needs the secret key.

Request (the fields I'd actually send)

```
{
  "amount": 4999,
  "currency": "GBP",
  "description": "Order #ORD-10234",
  "customer": {
    "email": "customer@example.com",
    "full_name": "Jane Doe"
  },
  "line_items": [
    {
      "name": "Wireless Mouse",
      "type": "physical",
      "quantity": { "value": 1 },
      "unit_price_amount": 4999,
      "total_amount": 4999
    }
  ],
  "capture_mode": "automatic",
  "merchant_order_data": {
    "reference": "ORD-10234"
  },
  "redirect_url": "https://mystore.com/checkout/success",
  "expire_pending_after": "PT30M"
}
```

Some things worth calling out here:

- **amount is always in minor units.** 4999 in GBP means £49.99. This catches people out constantly, especially with currencies like JPY or ISK that don't really have a minor unit at all (100 there just means 100 units, not 1.00). Check the currency's exponent before you hardcode any cent math.
- **line_items need to add up to amount exactly.** If you're doing tax or discount calculations, the sum of every line item's `total_amount` must match the order total, or the request just fails with a 400.
- **capture_mode: automatic vs manual is the biggest decision here.** Automatic capture takes the funds the second authorization succeeds. That's fine for digital goods, but risky for anything with a shipping delay, since you'd be holding the customer's money before you've actually sent anything. Manual capture authorizes and holds the funds, and you capture them yourself later (covered in section 4). For a store shipping physical items, I'd go with manual capture as the default.
- **merchant_order_data.reference is your own internal order ID.** Always set this. It's what you match against later when webhooks come in, since Revolut's order id means nothing to your existing systems.
- **expire_pending_after auto fails abandoned orders.** Without it, an order can sit in pending forever if someone opens checkout and just closes the tab.

Response, and which fields you actually need next

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "token": "ord_tok_9fc01989...",
  "state": "pending",
  "amount": 4999,
  "currency": "GBP",
  "outstanding_amount": 4999,
  "checkout_url": "https://checkout.revolut.com/pay/550e8400...",
  "created_at": "2026-08-03T14:22:00Z"
}
```

- **id.** The permanent order ID. Store this. Every operation after this point (capture, cancel, refund, retrieve) uses this ID.
- **token.** A temporary ID that expires once the payment gets authorized. This is what you'd hand to the frontend Merchant Web SDK if you're embedding checkout on your own site instead of redirecting. Since it expires, don't use it as a long term reference. Use id for that.
- **checkout_url.** If you're using the hosted checkout page (the redirect approach, which needs the least frontend work), this is the link you send the customer to.
- **state: pending.** Order created, no payment attempt yet. Write this to your orders table as soon as the response comes back.

After this, your backend's synchronous job is basically done. Save the order row, redirect the customer (or hand token to the embedded widget), and let webhooks handle the rest.

3. Payment processing (between order creation and completion)

The customer pays through whichever method you've enabled, card, Apple Pay, Google Pay, Revolut Pay, or Pay by Bank. No matter which one, the order moves through roughly this lifecycle:

```
pending -> processing -> authorised -> completed
                        \-> failed
authorised -> (manual capture) -> completed
authorised -> (cancel) -> cancelled
```

State	Meaning
pending	Order created, no successful payment attempt yet
processing	Payment submitted, being verified or authorized
authorised	Funds reserved on the card, not captured yet (manual capture only)
completed	Funds captured and settling to your account
cancelled	Order or authorization cancelled before capture
failed	Payment declined, or the order expired

Each order also has a payments array. A single order can have more than one payment attempt (say the customer's first card fails and they retry with another one). Each payment object has

its own, more detailed state (`authentication_challenge`, `authorisation_passed`, `declined`, and others), plus a `decline_reason` when it fails. There's a fixed list of about 25 decline reasons, things like `insufficient_funds`, `invalid_cvv`, `do_not_honour`.

3DS handling

If a payment's state is `authentication_challenge`, the response includes an `authentication_challenge` object with an `acs_url`. That's the bank's 3D Secure challenge page. If you're using the embedded Merchant Web SDK, it handles this redirect for you automatically. If you're calling the raw API yourself, you're the one responsible for sending the customer to `acs_url` and handling the return.

What's actually worth storing from a payment object

Once a payment settles, from the `payments[]` array, I'd persist:

- `id`, the payment ID (different from the order ID)
 - `state`, `decline_reason`, `bank_message`, useful for support and disputes later
 - `payment_method.type`, `card`, `apple_pay`, `google_pay`, `revolut_pay_card`, `revolut_pay_account`, or `sepa_direct_debit`
 - `payment_method.card_last_four`, `card_brand`, `card_expiry`, display safe card info, never the full card number
 - `payment_method.fingerprint`, a stable 44 character hash that identifies the payment instrument across transactions. Useful for fraud checks (blocking a card tied to past chargebacks) without ever storing the actual card number
 - `authorisation_code`, the issuer's approval code, needed if you ever have to manually reference a transaction with the bank
 - `network_transaction_id`, needed if you're linking retry or recurring attempts back to the original authorization
-

4. Capturing (manual capture mode)

Endpoint: `POST /api/orders/{order_id}/capture`

If `capture_mode` was set to `manual` at creation, the order sits in `authorised` state, holding the funds, until you call `capture` yourself. This call is idempotent. Calling it twice with the same amount just returns the current state instead of charging twice, but calling it twice with different amounts will error out. Good practice is to fire `capture` the moment your fulfillment pipeline confirms the order is actually going out, not before.

There's also a hard deadline here. The payment object has a `capture_deadline` field telling you when the issuer will release the hold automatically if you never capture. Don't leave authorized but uncaptured orders sitting around indefinitely.

5. Refunds

Endpoint: `POST /api/orders/{order_id}/refund`

```
{
  "amount": 4999,
  "currency": "GBP",
  "description": "Refund for returned item",
}
```

```

"merchant_order_data": {
  "reference": "REFUND-ORD-10234-1"
}
}

```

A few things to know about how this actually behaves:

- Refunds only work on orders in completed state. You can't refund a pending order, or one that's authorized but not yet captured. For those you'd cancel instead.
- currency has to match the original order's currency exactly.
- Partial refunds are allowed, and you can issue more than one against the same order as long as the total doesn't go over the original amount. That means your database needs to support several refunds per order, not just a single refund flag.
- The response is itself a new Order, with type: refund, and related_order_id pointing back to the original. It goes through the same lifecycle states (pending to completed) too. Refunds aren't instant either.
- **Always send an Idempotency-Key header on refund requests.** If a request times out and you retry it blindly, without an idempotency key you risk refunding the customer twice.

Idempotency-Key: refund-req-a3f9c2e1

I'd generate this key from your own refund record's primary key (or a UUID you save before firing the request), so retries reuse the same key even if your server restarts in between.

6. Webhooks

Polling for order status is a bad idea once you have any real volume. Webhooks are meant to be the source of truth for state changes.

Setup: POST /api/webhooks with a URL and the list of events you want (at minimum ORDER_COMPLETED and ORDER_AUTHORIZED, plus refund and dispute events if you support those). Revolut gives you back a signing_secret on creation. Store this somewhere safe, it's what lets you check that an incoming payload is really from Revolut and not something faked.

One thing to watch for: delivery order isn't guaranteed. For a normal successful payment you'd expect ORDER_AUTHORIZED before ORDER_COMPLETED, but if the first delivery attempt fails and gets retried, you might get ORDER_COMPLETED first. Your handler has to be idempotent and driven by state, not by assuming a particular order of events.

Verifying the signature (the part people skip, and then regret)

Every webhook POST comes with a Revolut-Signature header and a Revolut-Request-Timestamp header. You rebuild the signed payload and compare HMACs:

```

payload_to_sign = "v1." + timestamp + "." + raw_request_body
expected_signature = "v1=" + HMAC_SHA256(signing_secret, payload_to_sign)

```

Here's a Node/Express implementation:

```

const crypto = require('crypto');

function verifyRevolutWebhook(req, signingSecret) {
  const timestamp = req.headers['revolut-request-timestamp'];
  const receivedSignature = req.headers['revolut-signature'];
  const rawBody = req.rawBody; // must be the raw, unparsed body string

```

```

const payloadToSign = `v1.${timestamp}.${rawBody}`;
const expectedSignature = 'v1=' + crypto
  .createHmac('sha256', signingSecret)
  .update(payloadToSign)
  .digest('hex');

// timing safe comparison, avoids timing attacks
return crypto.timingSafeEqual(
  Buffer.from(expectedSignature),
  Buffer.from(receivedSignature)
);
}

```

Two things that will bite you if you're not careful:

1. **You need the raw body, not the parsed JSON.** If `express.json()` is running globally before your handler, the body's already parsed by the time you see it, and re-serializing it won't match byte for byte what Revolut actually signed (key order, whitespace, all of it). Use raw body middleware scoped just to the webhook route.
2. **Reject anything with an old timestamp**, a few minutes at most, so a valid signature from a captured request can't be replayed later.

If the signature doesn't match, reject with a 401. Don't process the event, don't return 200, don't log it and move on anyway.

7. PCI compliant database schema

The single biggest decision here: **your database should never hold a full card number, CVV, or any raw payment credential.** Revolut's checkout surface, hosted page or embedded widget, handles all of that for you. Card data never touches your server, which is what keeps you in the lightest PCI DSS tier (SAQ A) instead of the heaviest one (SAQ D). Everything below only stores references and display safe metadata.

```

-- Orders: mirrors Revolut's order lifecycle, keyed by our own PK plus their ID
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  reference VARCHAR(64) UNIQUE NOT NULL, -- our internal order ref, e.g. ORD-1
  revolut_order_id UUID UNIQUE NOT NULL, -- Revolut's `id`
  customer_id BIGINT NOT NULL REFERENCES customers(id),
  amount_minor BIGINT NOT NULL, -- always minor units, never a float
  currency CHAR(3) NOT NULL,
  state VARCHAR(20) NOT NULL, -- pending/processing/authorised/complete
  capture_mode VARCHAR(10) NOT NULL DEFAULT 'automatic',
  outstanding_amount_minor BIGINT,
  refunded_amount_minor BIGINT DEFAULT 0,
  checkout_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_orders_revolut_id ON orders(revolut_order_id);
CREATE INDEX idx_orders_state ON orders(state);

```

```

-- Payments: one order can have multiple payment attempts
CREATE TABLE payments (
  id BIGSERIAL PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(id),
  revolut_payment_id UUID UNIQUE NOT NULL,
  state VARCHAR(30) NOT NULL,
  decline_reason VARCHAR(50),
  bank_message TEXT,
  amount_minor BIGINT NOT NULL,
  currency CHAR(3) NOT NULL,
  payment_method_type VARCHAR(30), -- card / apple_pay / google_pay / revolut_pay_c
  card_brand VARCHAR(20), -- visa/mastercard/amex, display only
  card_last_four CHAR(4),
  card_expiry CHAR(5), -- MM/YY, display only
  payment_fingerprint CHAR(44), -- for fraud/dedup logic, not reversible to a
  authorisation_code VARCHAR(10),
  network_transaction_id VARCHAR(64),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_payments_order_id ON payments(order_id);
CREATE INDEX idx_payments_fingerprint ON payments(payment_fingerprint);

-- Refunds: kept as its own table, not a flag on orders, since partial refunds can stack
CREATE TABLE refunds (
  id BIGSERIAL PRIMARY KEY,
  original_order_id BIGINT NOT NULL REFERENCES orders(id),
  revolut_refund_order_id UUID UNIQUE NOT NULL, -- Revolut's refund order `id`
  amount_minor BIGINT NOT NULL,
  currency CHAR(3) NOT NULL,
  state VARCHAR(20) NOT NULL,
  reason TEXT,
  idempotency_key VARCHAR(64) UNIQUE NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_refunds_original_order ON refunds(original_order_id);

-- Webhook event log: replay protection, debugging, and an audit trail
CREATE TABLE webhook_events (
  id BIGSERIAL PRIMARY KEY,
  revolut_event_id VARCHAR(64), -- dedupe key if Revolut provides one, else hash o
  event_type VARCHAR(50) NOT NULL, -- ORDER_COMPLETED, ORDER_AUTHORIZED, etc.
  order_id BIGINT REFERENCES orders(id),
  raw_payload JSONB NOT NULL,
  signature_valid BOOLEAN NOT NULL,
  processed BOOLEAN NOT NULL DEFAULT false,
  received_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  processed_at TIMESTAMPTZ
);

CREATE INDEX idx_webhook_events_order_id ON webhook_events(order_id);

```

```
CREATE UNIQUE INDEX idx_webhook_events_dedupe ON webhook_events(revolut_event_id) WHERE revolut
```

A few design choices worth being able to explain out loud:

- **No card number, no CVV, anywhere in the schema.** Only `card_last_four`, `card_brand`, `card_expiry`, all display safe fields that come straight back from Revolut's own API response. We never capture card data through our own forms in the first place.
 - **payment_fingerprint lets us do fraud checks** (block a card that charged back before) without ever holding a reversible card number.
 - **Amounts are always BIGINT in minor units**, never FLOAT or anything derived from float math. Mixing floats with money is a classic way to get rounding bugs.
 - **webhook_events is append only**, kept separate from the order state itself. If a webhook handler crashes halfway through processing, we can replay from this table. It also gives us a paper trail for support disputes, like when a customer says they paid and we need to check whether we actually got confirmation.
 - **idempotency_key on refunds is unique at the database level**, not just passed as an API header. That way even a bug in the retry logic can't slip through and double refund someone.
-

8. Error handling

A few categories worth treating differently:

Validation errors (400). Malformed request, like line items that don't sum to the order amount, or missing address fields required for certain industries. These get caught before the request ever reaches a payment network. Fix the request and retry.

Authentication errors (401). Bad or expired secret key. This shouldn't happen in production unless a key got rotated and the deploy didn't pick up the new one. Worth alerting loudly on if it ever shows up.

Payment declines. Not really an API error at all. The request itself succeeds (201), but the payment inside that order comes back with `state: declined` or `failed`, along with a `decline_reason`. Map these to customer facing messages instead of showing the raw enum. `insufficient_funds` and `invalid_cvv` need very different copy than `suspected_fraud`, where you probably don't want to tell the customer the exact reason, for security purposes.

Network or timeout errors on your side. Retry idempotent GET requests freely. For POSTs that move money (creating an order, issuing a refund), only retry with an Idempotency-Key attached. A timeout followed by a blind retry can otherwise double charge or double refund someone.

Webhook delivery failures. Revolut retries failed deliveries on their own side, but your endpoint should still be defensive. Return 200 only once you've durably saved the event (write to `webhook_events` first, acknowledge second, and process the business logic asynchronously if it's heavy). If your endpoint returns a 500, that's fine, treat it as expected, the retry will come.

9. Security practices beyond just PCI scope

- **Secret key** goes in an environment variable or secrets manager, never in source control, never in a frontend bundle. Rotate it if you ever suspect a leak.

- **Webhook signature verification is not optional.** An unauthenticated webhook endpoint is basically an open door for someone to fake a “payment completed” event and trigger free fulfillment.
 - **TLS everywhere**, including the webhook receiver (Revolut requires HTTPS for webhook URLs anyway).
 - **Rate limit and check payload size** on the webhook endpoint before you even parse it, standard hardening against malformed or oversized requests.
 - **Idempotency keys on every money moving request** (orders, refunds), generated once per operation and reused on retries.
 - **Least privilege on the database.** If this is part of a bigger system, the service writing to payments and refunds shouldn’t have blanket access to every table.
 - **Log payment metadata, never payment credentials.** Even in error logs and stack traces, make sure full webhook payloads and customer PII are handled according to your data retention policy, not dumped into plaintext logs forever.
-

10. Assumptions I made

- Went with the **hosted checkout page (redirect) flow** as the main approach here, rather than the fully embedded Merchant Web SDK card field, since it needs the least custom PCI sensitive frontend code. Worth mentioning if the assessment actually expected the embedded widget instead.
 - Went with **manual capture mode** for this write up, on the reasoning that most stores shipping physical goods shouldn’t take the customer’s money before the order actually ships. Switching to automatic capture is a one line config change if the business is selling digital goods instead.
 - Used **GBP** in the examples. The minor unit handling generalizes to any ISO 4217 currency Revolut supports.
 - Assumed a **single Postgres database** for the schema. In a bigger production system, webhook_events might reasonably sit in its own append only store depending on volume.
 - Didn’t cover **subscriptions, payouts, or disputes**, since the brief scoped this to order creation, payment processing, and refunds.
-

References

- [Revolut Merchant API, Orders](#)
- [Create an order](#)
- [Refund an order](#)
- [Webhooks](#)
- [Verify the payload signature](#)